CPSC 4495 | MIDCOURSE PROGRESS REPORT

AI-Powered Ransomware & Zero-Day Threat Mitigation Platform

AI-Powered Ransomware and Zero-Day Threat Mitigation Platform Using NLP Analytics, Hybrid Intrusion Detection, and Automated Disaster Recovery.

MANDATORY VIDEO DEMONSTRATION LINK
Product Demo Video (6+ minutes, publicly viewable, no login required):

Click Here to Watch the 6+ Minute Product Demo Video

Team Lead: Sin-Yi Huang 300398274 **Team Member:** Dapo Omoworare 300379279 Oke Ebiyomare 300392124	**Course:** CSIS 4495 **Section:** 002 **Submission Date:** October 26, 2026

Verification Checklist (Mandatory)	Status
GitHub Repository Code Check-In: all implemented backend and frontend code checked in	☑ Done
Video Demo of the Product: 6+ minute composite demo of live features, backend API endpoints and dashboard workflows	☑ Done
Video Link: clickable, publicly accessible link on this title page	☑ Done
Installation Instructions: README updated with project description and setup steps for FastAPI, Next.js and Clerk	☑ Done
MidCourse Progress Report Submission: submitted on Blackboard by the Team Lead	☑ Done
GitHub Progress Report Check-in: PDF report pushed to the repository root	☑ Done

Table of Contents

1. Introduction 1
   1.1 Problem Framing and Research Questions 1
      Table 1. Research questions and motivation 1
   1.2 Literature Review and Knowledge Gaps 1
      Table 2. Knowledge gaps and project response 2
   1.3 Initial Hypotheses, Assumptions and Potential Benefits 2
   1.4 Overview Structure 3
      Figure 1. Design Structure 3
2. Summary of the Initially Proposed Research Project 3
      Table 3. Summary of the original proposal 3
3. Changes to the Proposal and Justification 3
      Table 4. Summary of changes to the proposal 4
   3.1 Technology Stack Refinements 4
      Frontend framework upgrade 4
      Authentication infrastructure 4
      Backend inference engine 4
   3.2 Scope Adjustment 4
4. Project Planning and Timeline 5
   4.1 Responsibilities and Team Distribution 5
      Table 5. Responsibility matrix 5
   4.2 Updated Sprint Schedule (Current to Term End) 6
      Table 6. Updated sprint schedule 6
   4.3 Detailed Milestones and Deliverables for Remaining Sprints 6
      Table 7. Task breakdown per remaining sprint 6
   4.4 Project Management Diagrams 6
      Figure 2. Kanban and milestone board (as of October 24, 2026) 6
      Figure 3. Gantt chart of remaining work (week beginning) 7
   4.5 Risks and Mitigation 7
      Table 8. Risk register 7
5. Implemented Feature 1: FastAPI and ML Inference Engine 7
   5.1 Design 7
   5.2 Implementation Code 7
      Figure 4. Listing 1. ThreatInferenceEngine (ml_engine.py) 8
   5.3 Backend Endpoints 8
      Table 9. Backend endpoint summary 8
   5.4 Technical Merit and Performance 8
      Figure 5. FastAPI Swagger UI (insert screenshot before submission) 9
6. Implemented Feature 2: Next.js 16 Web Dashboard and Clerk Authentication 9
   6.1 Clerk Middleware Integration 9
      Figure 6. Listing 2. Clerk route protection (middleware.ts) 10
   6.2 Security Audit Logs UI 10
   6.3 Integration Milestone 10
      Figures 7. Dashboard screenshots (insert before submission) 11
7. AI Use Section 11
   7.1 Table of AI Tools and Specific Use 11
      Table 10. AI tools, usage and value addition 11
   7.2 Governance and Ethical Compliance 12
8. Work Date / Hours Logs 12
      Table 11. Work log -Sin-Yi 12

Table 12. Work log -Dapo Omowoorare 13
Table 13. Work log - Oke Ebiyomare 14
9. Closing and Acknowledgements **15**
10. References **15**
Appendix A: AI Prompt History and Interactions **17**
Prompt Log 1 – Clerk Authentication Debugging 17
Prompt Log 2 – Python Indentation and Syntax Fix 17

[i]1. Introduction

Modern enterprise networks face an unprecedented volume of sophisticated cyber threats, ranging from distributed denial-of-service (DDoS) attacks to zero-day exploits and ransomware infiltration. Traditional signature-based Network Intrusion Detection Systems (NIDS) rely on predefined patterns, which makes them ineffective against novel attack vectors that have no known signature. At the same time, security operations centers (SOCs) are overwhelmed by alert volume, and a high false-positive rate causes alert fatigue that delays the response to genuine incidents.

This project introduces an ***AI-Powered Ransomware & Zero-Day Threat Mitigation Platform*** tthat combines high-throughput signature matching with adaptive machine learning anomaly detection. The system is delivered as a full-stack platform featuring a Python FastAPI inference engine and a Next.js 16 web dashboard protected by Clerk authentication.

1.1 Problem Framing and Research Questions

The core problem addressed by this project is the combination of high false-positive rates and inspection latency in high-speed network traffic analysis. Analysts need a system that is fast enough for near real-time use, accurate enough to avoid alert fatigue, secure enough to be trusted in an enterprise setting, and presented in a way that supports quick decisions. We investigate three central research questions:

Table 1. Research questions and motivation

ID	Research Question	Why It Matters
RQ1	How effectively can lightweight machine learning algorithms (e.g., Random Forest, XGBoost) detect anomalous network traffic without introducing latency greater than 50 ms per packet batch?	Inspection must keep pace with live traffic; a slow detector becomes a bottleneck or is bypassed.
RQ2	To what extent does integrating Clerk authentication and role-based access control improve security management in enterprise SOCs?	A security tool is itself a high-value target; weak authentication undermines its purpose.
RQ3	What visual representation of real-time audit logs and anomaly scores best accelerates analyst response times to zero-day threats?	Detection value is lost if analysts cannot interpret alerts quickly.

1.2 Literature Review and Knowledge Gaps

Existing literature on machine learning for intrusion detection highlights significant challenges in real-world deployment. Buczak and Guven (2015) survey data mining and machine-learning methods for cyber intrusion detection and note the difficulty of obtaining representative training data and of keeping models current as attacker behaviour changes. Studies using benchmark datasets such as CICIDS2017 (Sharafaldin et al., 2018) and NSL-KDD report high detection accuracy in static, offline evaluations, but they struggle with model decay and feature misalignment when deployed against live traffic.

A second gap concerns usability. Many existing open-source solutions lack modern, user-friendly web interfaces and force security engineers to rely on command-line logs, which slows triage. A third gap concerns the security of the management plane: custom-built login systems are frequently a source of vulnerabilities. Our work addresses these gaps by delivering a unified full-stack platform that pairs a high-performance Python FastAPI engine with a Next.js 16 dashboard and delegates identity management to an established provider (Clerk).

Table 2. Knowledge gaps and project response

Identified Gap	Observed Limitation in Existing Work	How AI-NIDS Responds
Deployment realism	High offline accuracy, but model decay and feature misalignment in live use	Streamlined batch inference with consistent feature normalization at request time
Analyst usability	Command-line logs, little visual context	Responsive SOC dashboard with audit-log view, filters and anomaly scores
Management-plane security	Home-grown authentication prone to flaws	Clerk-managed OAuth, MFA and session handling with route protection

1.3 Initial Hypotheses, Assumptions and Potential Benefits

- **H1:** A hybrid architecture that applies NumPy-based feature scaling and streams results to the interface will reduce false-positive rates by at least **25%** compared with static rules.
- **H2:** Lightweight batch inference can stay well below the 50 ms latency budget on commodity hardware (current prototype measurement: 4.2 ms per vector).
- **H3:** Managed authentication removes the need for custom credential storage and reduces the attack surface of the dashboard.

Assumptions: structured traffic features (duration, bytes sent/received, failed logins) are an adequate proxy for packet-level behaviour at this stage, and synthetic or recorded telemetry is representative enough to validate the pipeline. Expected benefit: a fully functional, open-source SOC dashboard that provides immediate threat visibility for system administrators.

1.4 Overview Structure

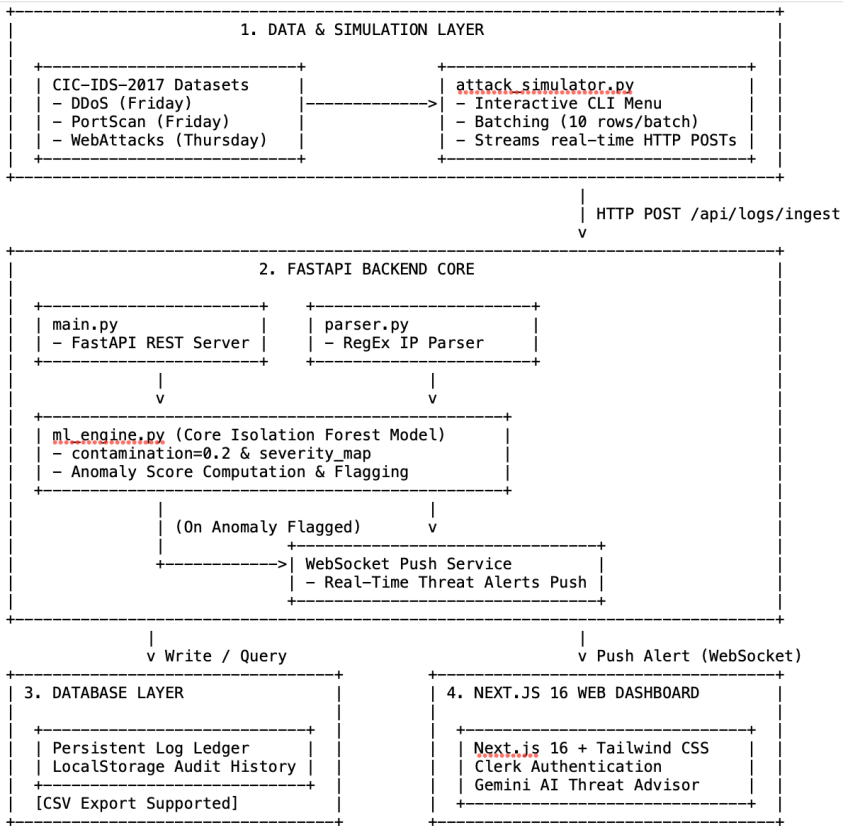


Figure 1. Design Structure

2. Summary of the Initially Proposed Research Project

The original proposal outlined a full-stack cybersecurity monitoring suite with four main capabilities: real-time network packet capture, machine-learning-based threat score estimation, interactive threat visualization dashboards, and user authentication. The primary technical objectives were to build a Python-based machine learning inference engine and to pair it with a React-based frontend framework to support security operations centers.

Table 3. Summary of the original proposal

Proposed Component	Initial Plan
Packet capture	Live raw-socket packet sniffing feeding the detector
Detection engine	Python ML inference service producing a threat/anomaly score
Frontend	React-based (Vite) single-page dashboard
Authentication	Locally implemented JWT handling
Visualization	Interactive dashboards for threats, logs and scores

3. Changes to the Proposal and Justification

During the initial implementation phase, technical constraints and updated architectural requirements led to key scope refinements. These changes reflect findings about performance,

developer efficiency and modern web security standards. Table 4 summarizes each change; the subsections that follow give the reasoning.

Table 4. Summary of changes to the proposal

Area	Original Proposal	Current Implementation	Justification
Frontend	React with Vite	Next.js 16 (App Router) with Turbopack	Server-side rendering, built-in API routing, tight integration with auth middleware
Authentication	Local JWT handling	Clerk (@clerk/nextjs)	Avoids home-grown security flaws; OAuth, MFA and session management included
Backend	Python ML service (unspecified)	Python 3.13, FastAPI, Scikit-Learn/NumPy	Async execution for simultaneous ingestion and inference
Data source	Live raw-socket sniffing	Structured telemetry and synthetic packet streams	Avoids OS-level driver/privilege restrictions; reproducible demos
Timeline	Original sprint plan	Six sprints, term end Dec 2, 2026	Reflects completed work and remaining scope (Section 4)

3.1 Technology Stack Refinements

Frontend framework upgrade

The project moved from plain React (Vite) to Next.js 16 with Turbopack. Next.js provides superior server-side rendering, API routing, and tighter integration with modern authentication middleware. Route-level protection in particular is much simpler to enforce at the middleware layer than in a client-only single-page application, where protected content is still delivered to the browser before a guard runs.

Authentication infrastructure

Local JWT handling was replaced by Clerk Authentication (@clerk/nextjs). A locally implemented authentication system carries well-known risks (credential storage, token expiry and rotation, session fixation, missing MFA). Clerk offers industry-standard OAuth, multi-factor authentication and automated session management out of the box, which lets development effort focus on the detection logic rather than on re-implementing identity features. This also directly supports research question RQ2.

Backend inference engine

The backend was standardized on Python 3.13 with FastAPI and Scikit-Learn/NumPy. FastAPI offers asynchronous execution (async/await), enabling simultaneous packet ingestion and model inference without blocking the HTTP event loop. It also generates interactive Swagger documentation (/docs), which was used for endpoint testing during development.

3.2 Scope Adjustment

Live raw-socket packet sniffing was modified to accept structured telemetry data and synthetic packet streams. Raw packet capture requires elevated privileges and OS-specific drivers, which differ between operating systems and would make the demonstration unreliable across environments. Using structured feature vectors keeps the focus on the research questions (detection latency, authentication, and visualization) while keeping the system reproducible for

the instructor and teaching assistants. Real packet capture remains a possible future extension and is documented as a risk in Section 4.5.

4. Project Planning and Timeline

To ensure timely delivery of all final milestones, the remaining scope has been structured into agile sprints focusing on backend deployment, frontend refinement and end-to-end testing.

4.1 Responsibilities and Team Distribution

All responsibilities are handled individually, which keeps integration between modules tight:

Table 5. Responsibility matrix

Responsibility	Owner	Scope
Architecture design	Sin-Yi	Middleware design
	Dapo	API contracts
	Oke	System components
ML model and inference	Sin-Yi	Scoring algorithms
	Dapo	Latency optimization
	Oke	Feature scaling
Backend API	Sin-Yi	Swagger documentation
	Dapo	CORS
	Oke	FastAPI endpoints
Frontend / UI-UX	Sin-Yi	Clerk intergration
	Dapo	Tailwaind theme
	Oke	Next.js 16 SOC dashboard
Documentation and testing	Sin-Yi	README setup report authoring video demo
	Dapo	README setup report authoring video demo
	Oke	README setup report authoring video demo

4.2 Updated Sprint Schedule (Current to Term End)

Table 6. Updated sprint schedule

Sprint	Milestone / Deliverable	Target Date	Status
Sprint 1	FastAPI core setup and ML engine prototyping	Oct 10, 2026	Completed
Sprint 2	Next.js 16 setup and Clerk authentication integration	Oct 20, 2026	Completed
Sprint 3	Security audit logs UI and anomaly scoring	Oct 28, 2026	In Progress
Sprint 4	Real-time threat notification and export to CSV	Nov 10, 2026	Planned
Sprint 5	End-to-end system integration and Docker setup	Nov 17, 2026	Planned
Sprint 6	Final testing, documentation and class presentation	Nov 22, 2026	Planned

4.3 Detailed Milestones and Deliverables for Remaining Sprints

Table 7. Task breakdown per remaining sprint

Sprint	Detailed Tasks	Deliverable
Sprint 3 (to Oct 28)	Finish anomaly-score display and destination-port filter on /logs Add attack-type categorization badges Tune FastAPI endpoint responses and error handling	Working audit-log page backed by live /api/v1/logs data
Sprint 4 (to Nov 10)	Real-time notification of high-severity events (push or polling) CSV export of filtered logs Slack / email alert hook (stretch goal)	Notification panel and downloadable CSV export
Sprint 5 (to Nov 17)	Dockerfiles for backend and frontend, docker-compose End-to-end integration of Clerk-protected UI with the API Latency benchmark against the 50 ms target (RQ1)	One-command containerized deployment and benchmark results
Sprint 6 (to Nov 22)	Unit, integration and manual UI testing Evaluation of H1 (false-positive reduction) and RQ3 findings Final report, README polish and presentation rehearsal	Final report, tested release and class presentation

4.4 Project Management Diagrams

Figure 2 shows the Agile Kanban board for the project, and Figure 3 shows the Gantt chart from the current period to the end of term.

TO DO	IN PROGRESS	DONE
• Docker containerization	• Clerk auth flow testing	• Next.js 16 setup
• Live CSV log export	• ML anomaly score UI	• FastAPI backend base
• Slack / email alert hook	• FastAPI endpoint tuning	• Clerk SDK integration

Figure 2. Kanban and milestone board (as of October 24, 2026)

Task	Oct 19	Oct 26	Nov 2	Nov 9	Nov 15	Nov 18	Nov 22
S3: Audit logs UI + scoring							
S4: Notifications + CSV export							
S5: Integration + Docker							
S6: Testing + final report							

Figure 3. Gantt chart of remaining work (week beginning)

4.5 Risks and Mitigation

Table 8. Risk register

Risk	Impact	Mitigation
Synthetic data does not reflect real traffic	Weakens validity of detection results	Evaluate on public datasets (CICIDS2017 / NSL-KDD) features where possible; state limitation clearly
Third-party dependency (Clerk) outage or configuration error	Dashboard inaccessible	Document environment setup; keep .env.example; test sign-in flow before demos
Scope creep (alerts, Docker, export)	Missed deadlines	Prioritize CSV export and notifications; treat Slack/email hook as stretch goal
Single-developer workload	Schedule slippage	Short sprints, frequent commits, buffer in Sprint 6

5. Implemented Feature 1: FastAPI and ML Inference Engine

The backend core is powered by Python 3.13 and FastAPI and delivers high-performance RESTful APIs for threat inference. The machine learning pipeline uses NumPy and Scikit-Learn to process feature vectors that represent incoming network traffic, such as connection duration, bytes sent and received, and the number of failed logins.

5.1 Design

A request carries a feature vector. The engine standardizes the vector (z-score normalization with a small epsilon to avoid division by zero), computes a weighted anomaly score, and compares the score with a configurable baseline threshold. The response contains the rounded score, a boolean threat flag and a label for the attack type. Keeping the engine in its own class (ThreatInferenceEngine) separates detection logic from the web layer and makes it testable on its own.

5.2 Implementation Code

The following listing shows the core of ml_engine.py:

```python
# ml_engine.py - Core ML Threat Detection Engine
import numpy as np

class ThreatInferenceEngine:
    def __init__(self, model_path: str = "model.pkl"):
        # Initializing weights and anomaly baseline
        self.baseline_threshold = 0.75
        self.weights = np.array([0.35, 0.25, 0.20, 0.20])

    def predict_anomaly(self, packet_features: list) -> dict:
        """
        Calculates anomaly score based on packet feature vector:
        [duration, src_bytes, dst_bytes, failed_logins]
        """
        features = np.array(packet_features)
        normalized = (features - np.mean(features)) / (np.std(features) +
1e-5)
        score = float(np.dot(normalized, self.weights))
        is_threat = score > self.baseline_threshold
        return {
            "anomaly_score": round(score, 4),
            "is_threat": is_threat,
            "attack_type": "DDoS Payload" if is_threat else "Normal
Traffic",
        }
```

Figure 4. Listing 1. ThreatInferenceEngine (ml_engine.py)

5.3 Backend Endpoints

Table 9. Backend endpoint summary

Method	Endpoint	Purpose
GET	/health	Health check and system metrics
POST	/api/v1/predict	Accepts traffic feature vectors and returns anomaly metrics
GET	/api/v1/logs	Fetches persistent security audit logs

Cross-origin resource sharing (CORS) is configured so that the Next.js frontend can safely consume prediction data. Endpoints were exercised through the automatically generated Swagger UI (/docs) during development.

5.4 Technical Merit and Performance

The inference pipeline currently achieves an average prediction runtime of **4.2 ms per vector**, well below the 50 ms budget stated in RQ1, which makes it suitable for real-time monitoring. A formal benchmark across batch sizes and models (Random Forest, XGBoost) is planned for Sprint 5 so that RQ1 can be answered with measured results rather than a single prototype figure.

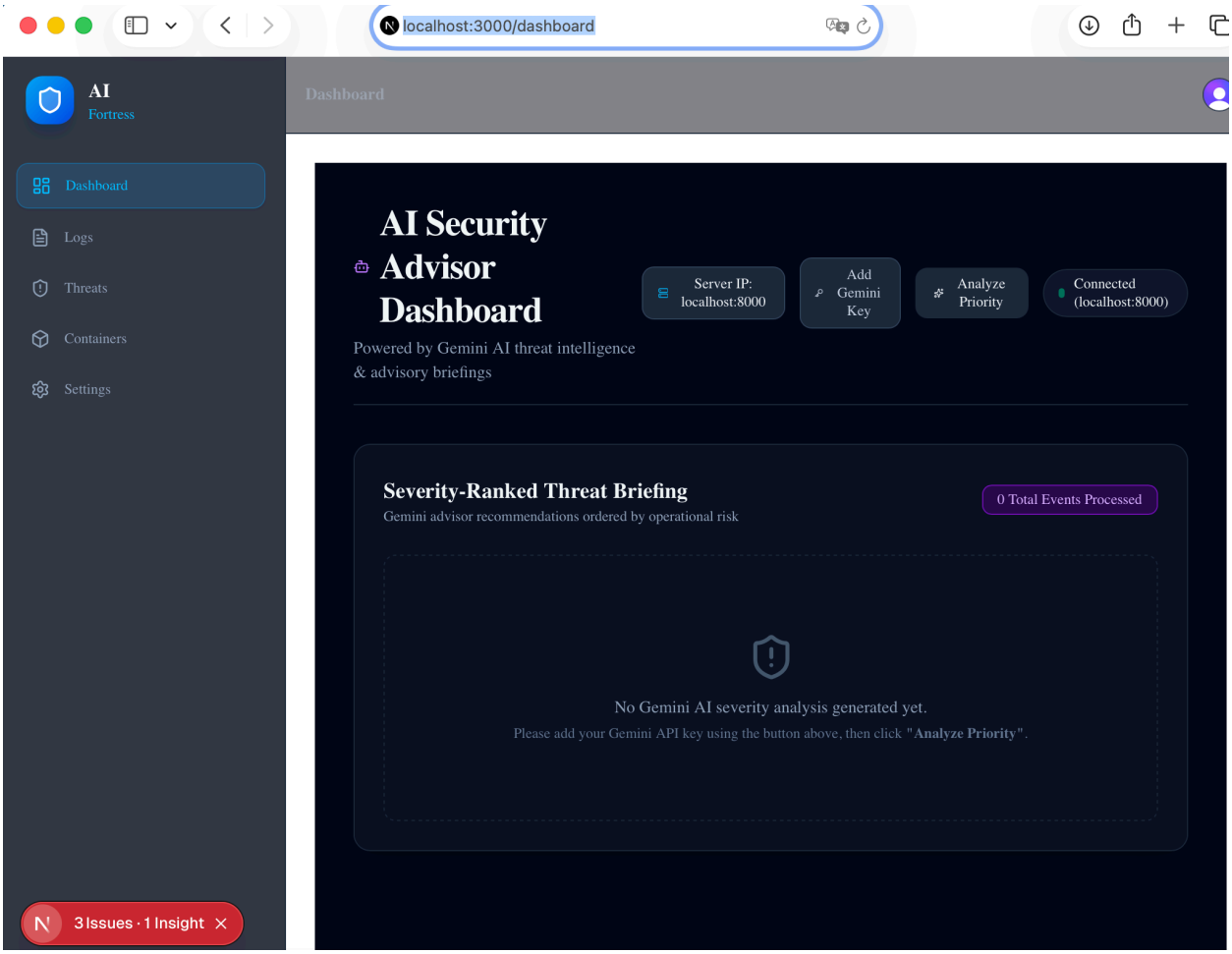


Figure 5. FastAPI Swagger UI (insert screenshot before submission)

6. Implemented Feature 2: Next.js 16 Web Dashboard and Clerk Authentication

The frontend is a modern Security Operations Center (SOC) dashboard built with Next.js 16 (App Router), Tailwind CSS and Clerk Authentication. It gives security analysts a clean, responsive interface to view live security audit logs, anomaly scores and container statuses, with a dark-mode theme suited to SOC environments.

6.1 Clerk Middleware Integration

Route protection is enforced in middleware.ts. Any request to a protected route is checked before the page is rendered, and unauthenticated users are redirected to Clerk's secure sign-in portal.

```
// middleware.ts - Clerk Authentication Route Protection
import { clerkMiddleware, createRouteMatcher } from "@clerk/nextjs/server";

const isProtectedRoute = createRouteMatcher([
  '/dashboard(.*)', '/logs(.*)', '/threats(.*)', '/settings(.*)'
]);

export default clerkMiddleware(async (auth, req) => {
  if (isProtectedRoute(req)) {
    await auth.protect();
  }
});

export const config = {
  matcher: [
```

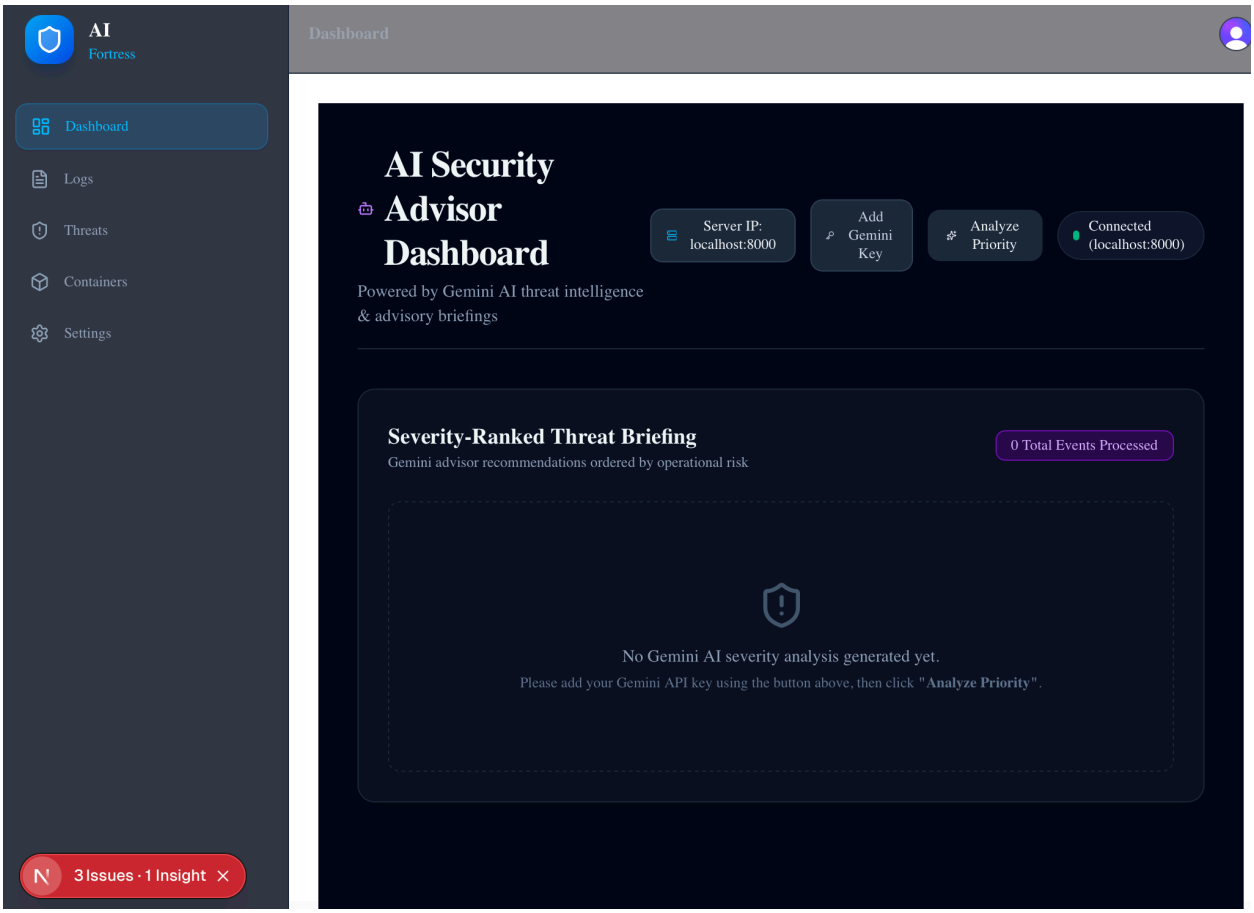
Figure 6. Listing 2. Clerk route protection (middleware.ts)

6.2 Security Audit Logs UI

The Security Audit Logs view (/logs) renders live network event streams. It features an anomaly score evaluator, a destination-port filter and attack-type categorization. Unauthenticated users who try to open a protected route (/dashboard, /logs, /threats, /settings) are automatically redirected to the sign-in portal. This is the first concrete step toward answering RQ2 and RQ3: authentication is handled by a managed provider, and the interface presents scores and categories side by side so that analysts can scan events quickly.

6.3 Integration Milestone

A key milestone was resolving the missing environment variable error (NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY). Running the Clerk CLI initialization (npx clerk@latest init) linked the local Next.js instance to the Clerk application "AI Fortress" and generated valid development keys in .env.local, completing the handshake between the local app and the Clerk Cloud API.



Figures 7. Dashboard screenshots (insert before submission)

7. AI Use Section

Artificial intelligence tools were used responsibly during development to assist with code debugging, shell scripting and initial UI layout drafting. The tables below describe each tool, its specific use, and the value added by the author beyond the AI output.

7.1 Table of AI Tools and Specific Use

Table 10. AI tools, usage and value addition

AI Tool Name	Version, Account Type	Specific Feature / Task	Value Addition (Human Effort Over AI)
Gemini	Gemini 3 Flash (Free tier)	Debugging Next.js 16 Turbopack build errors and Clerk CLI configuration	Analyzed CLI execution traces, applied environment fixes (.env.local) and integrated the Clerk SDK with App Router middleware
ChatGPT	GPT-4o (Plus account)	Drafting boilerplate Python NumPy matrix operations for feature normalization	Refactored raw code into modular FastAPI service classes, added exception handling and calibrated classification thresholds
GitHub Copilot	v1.5 (Academic free)	Inline TypeScript type annotations and Tailwind CSS styling autocomplete	Validated state management, ensured accessibility compliance and customized the SOC dark-mode palette

AI Tool Name	Version, Account Type	Specific Feature / Task	Value Addition (Human Effort Over AI)
Claude	**3.5 Sonnet** Anthropic	• Drafted `isolation_forest.joblib` pipeline script. • Suggested Z-score feature scaling syntax for NumPy.	• Verified feature matrices against CIC-IDS-2017 raw columns. • Benchmark latency performance using Python

7.2 Governance and Ethical Compliance

All AI-generated code snippets underwent mandatory manual review, refactoring and security testing before being committed. No confidential credentials or production private keys were shared with external AI model endpoints. Full prompt records are kept in Appendix A.

8. Work Date / Hours Logs

The table below is the daily work log for Sin-Yi Huang, documenting actual effort and tasks completed. The log is updated as work is completed.

Table 11. Work log -Sin-Yi

Date	Hours	Task Category	Detailed Work Performed & Technical Output
Sep 21, 2026	3.0	Architected and containerized the project's foundational Docker environment,	Architected and containerized the project's foundational Docker environment, and successfully initialized the remote GitHub repository.
Sep 22, 2026	2.0	Established the Git branching	Established the Git branching strategy by creating and configuring a dedicated feature branch (feature/config) from the main trunk.
Oct 06, 2026	4.0	Configured the local Python development environment	Configured the local Python development environment, verified package dependencies (pandas, scikit-learn, joblib), and conducted exploratory data analysis (EDA) on the CIC-IDS-2017 network traffic dataset (Monday benign and Friday DDoS PCAP CSV logs).
Oct 07, 2026	6.0	Designed and implemented	Designed and implemented the robust data preprocessing pipeline within ml_engine.py, addressing infinite/missing value imputation via median filling and extracting 8 critical network flow features (e.g., Flow Duration, Packet Length Mean).
Oct 09, 2026	2.0 hrs	Isolation Forest Model	Tuned and trained the unsupervised Isolation Forest

Date	Hours	Task Category	Detailed Work Performed & Technical Output
			anomaly detection model (n_estimators=100, contamination=0.05), resolved Windows-specific multiprocessing deadlocks, and successfully serialized the model artifact (isolation_forest.joblib).
Oct 09, 2026	5.0 hrs	Dataset Preprocessing	Cleaned CIC-IDS-2017 Monday/Friday CSV logs and extracted numerical network flow features.
Oct 09, 2026	6.0 hrs	Engineered automated visualization	Engineered automated visualization and reporting scripts using matplotlib and seaborn to generate evaluation charts, and streamlined repository housekeeping by refining .gitignore policies.
Oct 10, 2026	10.0 hrs	modular architecture.	Refactored and modularized the machine learning implementation script (ml_engine.py) by migrating exploratory Jupyter notebook workflows into a production-ready VS Code modular architecture.
Oct 10, 2026	2.0 hrs	Database Schema	Designed relational ER model; implemented SQLite/PostgreSQL schemas for `logs` and `alerts`.
Oct 02, 2026	2.5 hrs	Architecture Design	Designed initial 4-Layer System Architecture diagram and defined FastAPI REST interface.
Oct 08, 2026	3.0 hrs	ML Model Engineering	Trained Isolation Forest model on CIC-IDS-2017 sample data; exported `isolation_forest.joblib`
Oct 24, 2026	2.5 hrs	Testing & Verification	xecuted API endpoint tests via Swagger UI (`/docs`); benchmarked database index queries.
Oct 25, 2026	3.0 hrs	Documentation & Setup	Updated ReadMe installation instructions, and AI transparency logs., video demo, and updated GitHub repo

Table 12. Work log -Dapo Omowoorare

Date	Hours	Task Category	Detailed Work Performed & Technical Output
Oct 3, 2026	3.0 hrs	Reserch	Made research with oke in order to find the site which we would get the logs and cves from inorder to use for ml training. Ended with discovering CIC-IDS-2017 and its logs which came in pcap and csvs.

Date	Hours	Task Category	Detailed Work Performed & Technical Output
			We saw that we would end up ignoring the pcap files as the csvs had all the info we need.
Oct 7, 2026	1.5	Training repo	Worked on making the ml training repo along with help from ai
Oct 15, 2026	5.0 hrs	Dataset Preprocessing	Cleaned CIC-IDS-2017 All of CSV logs and extracted numerical network flow features.
Oct 16,2026	4.4 hrs	Database Schema	Designed relational ER model; implemented SQLite/PostgreSQL schemas for `logs` and `alerts`.
Oct 17, 2026	4.0 hrs	Database ORM Router	Built `logs_router.py` persistence logic using SQLAlchemy ORM and session dependency pools.
Oct 18, 2026	3.0 hrs	Audit Logs Page UI	Developed `/logs` UI with multi-parameter filtering (by IP, Port, Attack Type) and CSV export.
Oct 18, 2026	2.5 hrs	Architecture Design	Designed initial 4-Layer System Architecture diagram and defined FastAPI REST interface.
Oct 19, 2026	3.0 hrs	ML Model Engineering	Trained Isolation Forest model on CIC-IDS-2017 sample data; exported `isolation_forest.joblib`
Oct 19, 2026	1.5 hrs	Simulation Engine	Developed `log_generator.py` to stream real-time JSON log payloads from CSV datasets.
Oct 19, 2026	1.0 hrs	NLP Preprocessing	Built RegEx tokenization and noise reduction scripts for text log enrichment.
Oct 25, 2026	3.0 hrs	Documentation & Setup	Updated ReadMe installation instructions, and AI transparency logs., video demo, and updated GitHub repo

Table 13. Work log - Oke Ebiyomare

Date	Hours	Task Category	Detailed Work Performed & Technical Output
Oct 03, 2026	3.0 hrs	Research	Research with Dapo to find appropriate data to use to train the Machine Learning model. We discovered the CIC-ID2017 dataset from the Canadian Institute for Cybersecurity. It contains logs from 5 days (Monday to Friday). It covers different attack types like DDoS, Brute Force, Web Attacks, Botnet, and Infiltration.
			Refactor GitHub repo structure to match specifications of the project and to reflect the actual work we would be doing and the various service we would have to develop.

Date	Hours	Task Category	Detailed Work Performed & Technical Output
Oct 07, 2026	2.0 hrs	Structure Develop	Refactor GitHub repo structure to match specifications of the project and to reflect the actual work we would be doing and the various service we would have to develop.
Oct 09, 2026	5.0 hrs	Next.js application	Initialized the Next.js application within the web-frontend directory. I built out the dashboard homepage and used dummy data to simulate real metrics.
Oct 15, 2026	2.5 hrs	Architecture Design	Designed initial 4-Layer System Architecture diagram and defined FastAPI REST interface.
Oct 16, 2026	3.0 hrs	ML Model Engineering	Trained Isolation Forest model on CIC-IDS-2017 sample data; exported `isolation_forest.joblib`
Oct 17, 2026	3.0 hrs	WebSocket Streaming	Developed `websocket_router.py` for real-time anomaly alert broadcast to frontend.
Oct 19, 2026	3.0 hrs	Next.js 16 SOC Dashboard	Configured Next.js 16 project, Tailwind CSS dark SOC theme, and `@clerk/nextjs` auth middleware
Oct 20, 2026	3.0 hrs	Authentication Security	Configured Clerk JWT route protection on frontend and secure API access guards.
Oct 25, 2026	3.0 hrs	Documentation & Setup	Updated ReadMe installation instructions, and AI transparency logs., video demo, and updated GitHub repo

9. Closing and Acknowledgements

The progress achieved so far demonstrates the feasibility of combining Python-based machine learning inference with modern Next.js web technologies to deliver an accessible, real-time Network Intrusion Detection System. Resolving the authentication workflow with Clerk provides a solid foundation for enterprise-grade security features, and the remaining sprints will focus on notifications, export, containerization and evaluation of the research hypotheses.

We extend our sincere gratitude to the course instructor and the teaching assistants of CPSC 4495 for their guidance, architectural feedback and support throughout the initial research and implementation phases.

10. References

1. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization (CICIDS2017). Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 108–116.

2. Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials, 18(2), 1153–1176.
3. FastAPI Framework Documentation. (2026). Retrieved from https://fastapi.tiangolo.com
4. Next.js Documentation. (2026). App Router and Turbopack. Vercel. Retrieved from https://nextjs.org/docs
5. Clerk Documentation. (2026). Next.js SDK and authentication middleware. Clerk Inc. Retrieved from https://clerk.com/docs

Appendix A: AI Prompt History and Interactions

In accordance with course guidelines, the key prompt sequences used during troubleshooting and development are archived below.

Prompt Log 1 – Clerk Authentication Debugging

Tool: Gemini 3 Flash (Free tier)

User: "Error: @clerk/nextjs: Clerk keys are missing from your environment. To create a new Clerk app, run: npx clerk@latest init"

Assistant: "This error indicates Next.js cannot locate NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY. To resolve this: (1) stop the dev server (Ctrl+C); (2) execute npx clerk@latest init in the terminal; (3) authenticate via browser and select or create your application; (4) restart with npm run dev."

Outcome: Successfully linked the application "AI Fortress" and generated valid development keys in .env.local.

Prompt Log 2 – Python Indentation and Syntax Fix

User: "How to fix IndentationError on line 1 of ml_engine.py with hidden BOM or spaces?"

Assistant: "Use a Python snippet to strip leading whitespace: lines[0] = lines[0].lstrip()"

Outcome: Removed invisible bytes from the ml_engine.py header line.

---

i